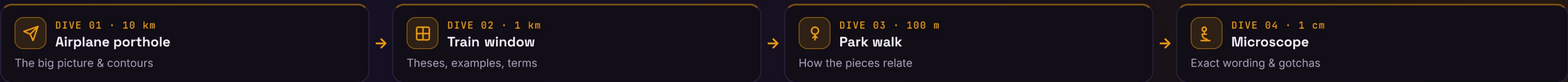




Memento

Capture and externalize an object’s internal state so it can be restored later — without ever breaking its encapsulation.



 Airplane porthole

DIVE 01 · ORIENTATION

▼ 10 km

A behavioral pattern for undo — it externalizes a snapshot of private state so the object can be rewound later, with encapsulation untouched.

— THE CONTOURS

• Saves state without exposing it
— internals stay private to the originator

• Originator owns save & restore
the caretaker just holds opaque tokens

• Powers undo / rollback,
at the cost of snapshot memory

— USE IT WHEN — GoF applicability

 A snapshot of an object's state
must be saved so it can be restored to that state later

 A direct interface would expose
implementation details and break the object's encapsulation

 Behavioral family: Command · Observer · State · Strategy · Memento — Memento freezes a snapshot of state; Command records the operation that changed it.

↓ DIVE DEEPER

 Train window

DIVE 02 · KEY OBJECTS

▼ 1 km

— THE THREE PARTS

Originator
creates a memento of its state and restores itself from one

Memento
stores the snapshot — wide interface for originator, narrow for caretaker

Caretaker
keeps mementos, never reads or mutates their contents

— IN THE WILD

 Redux DevTools
time-travel debugging steps through state snapshots

 SQL SAVEPOINT
partial rollback to a named in-transaction checkpoint

 Git
each commit is a restorable snapshot of the tree

 java.io.Serializable
serialized object graph is a persistent memento

<> Caretaker → Originator.save() ⇒ Memento → Originator.restore(m) rewinds state.

| GoF intent: “Without violating encapsulation, capture and externalize an object’s internal state so that the object can be restored to this state later.”

↓ DIVE DEEPER

 Park walk

DIVE 03 · CONNECTIONS

▼ 100 m

User edits
state mutates

→

save()
snapshot taken

→

push to history
caretaker stores it

→

✓ undo → restore(m)
state rewound

— WHY IT FOLLOWS

→ Wide vs narrow interface ⇒ originator reads everything, caretaker sees an opaque token

→ Caretaker is dumb ⇒ it stacks mementos but never inspects them — encapsulation holds

→ Restore is total ⇒ applying a memento returns the originator to that exact past state

 Often paired with Command for undo stacks · Iterator can capture position · Prototype clones a copy where deep copy is acceptable.

— STRUCTURE · UML

Originator

- state: State

+ save(): Memento

+ restore(m: Memento): void

save() packs state into a Memento «opaque to caretaker»; restore() reads it back.

— WHAT IT BUYS YOU

✓ Preserves encapsulation — private state leaves the originator only as an opaque memento

✓ Simplifies the originator — clients keep the version history, not the object itself

✓ Clean undo / rollback — restoring is just handing back a previously taken snapshot

✓ Memory cost — full snapshots can be expensive if state is large or saved often

✓ Caretaker bookkeeping — someone must cap, age out & track the lifecycle of mementos

↓ DIVE DEEPER

 Microscope

DIVE 04 · DETAILS

▼ 1 cm

| Definition: “Without violating encapsulation, capture and externalize an object’s internal state so that the object can later be restored to this state.”
— GoF, “Design Patterns”, 1994

— THE CODE

```
class Memento {
  constructor(readonly state: string) {}
}
class Editor {
  private text = "";
  type(s: string) { this.text += s; }
  save() { return new Memento(this.text); }
  restore(m: Memento) { this.text = m.state; }
}
// Caretaker keeps the stack, never reads
// m.state — encapsulation stays intact.
```

— COMPARE THE ALTERNATIVES

Property	Memento	Command	Serialization	Event sourcing
Restores past state	✓	~	✓	✓
Preserves encapsulation	✓	✓	✗	~
Stores deltas not snapshots	✗	✓	✗	✓
Built-in redo	~	✓	✗	✓
Persistable to disk	~	~	✓	✓

Best fit → Memento: in-memory undo · Command: action log/redo · Serialization: persistence · Event sourcing: audit + replay. (“~” = partial.)

— INTERVIEW PITFALLS

Q “How does it preserve encapsulation?”
The memento exposes a wide interface only to the originator; the caretaker holds an opaque token it can't read or change.

Q “Memento vs Command for undo?”
Memento snapshots the resulting state; Command records the operation. Command supports redo and is lighter; Memento is simpler for complex state.

Q “Deep or shallow copy?”
Deep enough that later mutations can't reach into the saved state — otherwise restore brings back already-corrupted data.

Q “How do you bound memory?”
Cap the history, store diffs instead of full snapshots, use copy-on-write, or persist older mementos out of memory.

— VARIANTS

Full snapshot
copy the whole state — simple, but heavy on memory

Incremental / diff
store only the delta since the last memento

Command + Memento
undo stack pairs each action with its snapshot

Copy-on-write
share state until a write forks a new version cheaply

Serialized / persistent
write the memento to disk or a database

Immutable snapshot
frozen value objects — Redux-style state history

— NUANCES & GOTCHAS

• Memory pressure — full snapshots of large state cost real memory; prefer diffs or copy-on-write when histories grow

• Deep vs shallow copy — a shallow snapshot still shares mutable references, so later edits silently corrupt the past

• Narrow vs wide interface — only the originator gets full access; the caretaker sees an opaque token, which is what preserves encapsulation

• Caretaker must not mutate — if it edits or reuses a memento, restore no longer returns the original state

• Persistence concerns — serialized mementos need versioning & schema migration as the state shape evolves

• Unbounded growth — an undo stack with no cap leaks memory; bound it or age old mementos out

— WATCH OUT

 Anti-pattern: Skip it when state is trivial to recompute or already immutable, or when snapshots are so large/frequent that the memory cost outweighs the undo benefit — log the operations (Command) or use event sourcing instead.

 Modern take: In practice undo is often built on immutable state plus structural sharing (Redux, Immer) — a persistent data structure is one big memento, and time-travel comes almost for free.

• Vasyi Krupka · Senior Fullstack Engineer · learn-by-diving series

Source: GoF “Design Patterns” (1994)

v1